Phase 1 Outline: Problem Formalization & Literature Review

Intelligent Load-Shedding Scheduler for Power Grids — Weeks 1–2

---

0. Phase 1 at a Glance

Duration	Team	Core Deliverables	Exit Criteria (from master plan)
Weeks 1–2	Parth – Team Lead, Mitali, Mehak	Formal CSP definition · related-work table (industry + academic) · dataset selection (data-sources.md)	Formal spec reviewed & approved by all 3; data-sources.md lists ≥4 verified, accessible sources

This document breaks that one line from the Phase 8 timeline table into something actually runnable by three people over ten working days.

---

1. Problem Statement (Phase-1 self-contained version)

The handout's Section 1 already nails the input and the output — it's the process/methodology step that's implicit rather than stated. Below is the same problem, restated once, in one place, under all four headings, so this document doesn't require flipping back to the handout to be understood on its own.

1.1 Context — why this problem exists

Indian power discoms routinely face demand-supply gaps during heatwaves, festival-season spikes, or equipment failures. When that happens, an operator manually picks which feeders to cut — under time pressure, with no principled way to balance fairness (who gets cut, how often), safety (never cut hospitals/water systems), and minimal disruption. This is a well-known, recurring, public-facing failure mode with real equity consequences (the same localities losing power repeatedly) and real safety consequences (critical infrastructure occasionally shed by human error).

1.2 Input — what the system is given

- A shortfall value S (kW) that must be resolved over a time horizon T — arriving as a live value from an operator or a pluggable forecaster; the system consumes S, it does not predict it.
- A grid modelled as feeders F = {f1, ..., fn}, each with a load capacity, a priority class (protected / non-critical), and a shedding history (downtime ledger).
- A policy set: max consecutive downtime per feeder, a fairness bound over a rolling window, and harm-cost weights (disruption × priority × duration).

1.3 Methodology / Process — how the system gets from input to output

This is the step that was previously spread across Sections 4–6 of the handout. Stated as one process, it's a five-stage pipeline that repeats on a rolling cadence:

1. Compile: current grid state + policy → a CSP instance (variables = feeder × time-slot, domain = {on, off}).
2. Solve: attempt backtracking + arc-consistency first (exact); fall back to min-conflicts local search at scale, or when a time budget is tight.
3. Relax (only if infeasible): drop soft constraints in a fixed order — fairness bound, then max-downtime, then non-critical priority class — never the protected-feeder rule — and log exactly what was relaxed and why.
4. Act: issue the resulting cut/restore schedule to the (simulated) actuation layer; log outcomes back to the grid state store.
5. Replan: re-trigger stage 1 on a fixed cadence or whenever the shortfall changes past a threshold, using the updated downtime ledger — this is what makes fairness-over-time and max-downtime real constraints instead of one-shot approximations.

Everything downstream (Section 6 syllabus mapping, Section 9 evaluation) is this five-stage loop viewed from a different angle — the CSP unit maps to stages 1–3, the agents unit maps to stage 5, and so on.

1.4 Expected Output

- A per-feeder, per-time-slot cut schedule (on/off + duration) that resolves the shortfall in every slot.
- A relaxation log (empty in the feasible case) stating exactly which soft constraint was relaxed, by how much, and why.
- A fairness score (Gini or max-to-mean ratio of cut-time) computed every replan cycle.
- A natural-language explanation, on request, grounded strictly in the above — never a free-floating justification.

---

2. Phase 1 Scope — What This Phase Is and Isn't

In scope:

- Writing down the CSP formally enough that Phase 3–4 can implement against it without re-litigating definitions.
- Confirming every dataset we plan to use actually exists, is accessible, and has the fields we think it has.
- Positioning the project against both industry systems and academic CSP-scheduling literature, so Section 3 of the handout has citations instead of assertions.

Out of scope (deliberately, for Phase 1):

- Any code. Phase 1 produces documents and a validated data plan, not a running generator or solver — that starts Phase 2.
- Deciding the exact fairness metric formula or harm-cost weights — those get tuned empirically starting Phase 4; Phase 1 only needs to name the candidates and how we'll choose.

---

3. Team Roles & Ownership

Assigned by actual strengths rather than generic labels — this replaces the placeholder "Person A/B/C" from the master plan with real ownership.

Person	Background	Phase 1 Ownership
Parth (Team Lead)	Backend, API design, knowledge graphs, AI @ JNU	Formal CSP definition (Section 1 above, expanded to a spec doc); scoping the KR&R rule base (protected-load rules, relaxation order) as a structured/graph-like representation, playing to the knowledge-graph background; final integration and review of all Phase 1 outputs before sign-off
Mitali	Frontend, UI/UX, EDA	Owns dataset acquisition + exploratory data analysis on all four confirmed sources (Grid-India/POSOCO, Kaggle compilation, hourly load dataset, data.gov.in); produces data-sources.md with real field lists, access notes, and license terms; first pass on what a synthetic grid topology needs to look like structurally, informed by what the real data shows
Mehak	GenAI, UI, some backend, strong PM instincts	Owns the literature review — both the industry-approaches table (already drafted) and the academic CSP-scheduling lineage (exam timetabling / bed allocation / crew scheduling) that Section 3 now references but doesn't yet cite; owns Phase 1 tracking (task board, daily standup notes, blocking-issue escalation); writes an early scoping note for the LLM explanation layer's grounding strategy (light touch — full design isn't until Phase 9, but the trace format it will need should be flagged now, before Parth finalizes the spec)

Cross-checks: every deliverable below gets reviewed by the other two before it's marked done — that review is what "reviewed and approved by all 3" in the exit criteria actually means, not a rubber stamp on the last day.

---

4. Task Breakdown

4.1 Formal CSP Definition — Owner: Parth

- Write the CSP spec doc: variables, domains, hard constraints, soft constraints, objective function — matching Section 1.2–1.3 above and Section 6 of the handout exactly, so there's one canonical definition instead of three slightly different ones across documents.
- Draft the KR&R rule-base structure for protected-load rules and the relaxation order (fairness → downtime → priority, never protected) — sketch it as a small inspectable rule set, not hardcoded logic, since this doubles as Unit 4 (KR&R) content per Section 6.
- Write the one-paragraph A* scope note from Section 6 into the spec doc directly, so Phase 5 (A* baseline) doesn't accidentally re-open the "is this comparable to the CSP" question.
    - Deliverable: CSP-spec.md
    - Due: end of Week 1 (draft) → reviewed Week 2 (final)

4.2 Data Acquisition & EDA — Owner: Mitali

- Confirm access to all four sources named in handout Section 7: Grid-India/POSOCO, the Kaggle demand/shortage compilation, the hourly load dataset, data.gov.in — pull a sample from each, don't just link to a listing page.
- For each source, document: fields available, granularity (hourly? state-level? feeder-level?), date range, license/usage terms, and known gaps.
- Run a first EDA pass to check that a realistic mild/moderate/crisis shortfall range can actually be derived from this data (needed before Phase 2's scenario generator can be built against it).
- Flag anything that doesn't hold up — e.g., a dataset that turns out to be state-level only when feeder-level granularity was assumed — before Phase 2 starts, not during it.
    - Deliverable: data-sources.md + a short EDA notebook/summary
    - Due: first pass Week 1, finalized Week 2

4.3 Literature Review — Owner: Mehak

- Finalize the industry-approaches table (manual rotational scheduling, pure ML forecasting, greedy SCADA heuristics, commercial DERMS/ADMS) with at least one real citation per row.
- Research and add the academic CSP-scheduling lineage row: find 2–3 real papers/references on exam timetabling, hospital bed allocation, or crew scheduling as CSPs, and note specifically what constraint-modeling pattern each contributes (this is what makes the new Section 3 row defensible in a viva, not just a claim).
- Write the short LLM-explanation-layer grounding scoping note — what should the constraint trace log look like structurally so an LLM can answer from it later — and hand it to Parth before the CSP spec is finalized, so the trace format is designed once, correctly, up front.
- Own the Phase 1 task board and run two short check-ins (mid-Week 1, mid-Week 2) to catch blockers before they cost a week.
    - Deliverable: related-work.md (industry + academic) + LLM-trace scoping note
    - Due: draft Week 1, finalized Week 2

---

5. Two-Week Schedule

	Parth	**Mitali**	**Mehak**
Days 1–2	Draft CSP variables/domains/constraints	Identify & request access to all 4 data sources	Draft industry-approaches table with citations
Days 3–5	Draft KR&R rule-base structure	Pull samples; document fields/granularity/license per source	Research academic CSP-scheduling lineage (2–3 refs)
Day 5 (checkpoint)	Share CSP-spec draft v0.1 with team	Share data-sources.md draft v0.1	Mid-Week 1 check-in: log blockers

	Parth	Mitali	Mehak
Days 6–8	Incorporate feedback; write A* scope note	Run first EDA pass; validate shortfall-range assumptions	Write LLM-trace scoping note; hand to Parth
Days 9	Finalize CSP-spec.md	Finalize data-sources.md + EDA summary	Finalize related-work.md
Day 10 (checkpoint)	All 3: joint review & sign-off session	All 3: joint review & sign-off session	All 3: joint review & sign-off session

---

6. Definition of Done — Phase 1 Exit Checklist

- CSP-spec.md exists, is internally consistent with handout Sections 1 and 6, and all 3 members have reviewed and approved it.
- data-sources.md lists ≥4 verified, accessible sources with real field lists, granularity, and license terms — not placeholder links.
- related-work.md covers both industry approaches and academic CSP-scheduling literature, each row backed by a real citation.
- A short EDA summary confirms the mild/moderate/crisis shortfall scenario ranges planned for Phase 2 are grounded in real data, not guessed.
- The LLM-trace scoping note exists and has been read by Parth before CSP-spec.md is marked final (so the trace format doesn't need rework in Phase 9).
- A joint review session has happened where all three walk through the CSP spec together — not an async "looks good" — since this is the spec Phases 3–6 will build against for the next 9 weeks.

---

7. Phase-1-Specific Risks

- Dataset access delays: government/portal data sources can be slow to register for or rate-limited. Mitali should attempt access on Day 1, not Day 5, so a blocked source can be swapped before it costs the phase.
- Spec churn: if the CSP spec changes materially after Phase 3 has started coding against it, that's expensive. The Day 10 joint review exists specifically to catch disagreements now, cheaply, rather than in Week 6.
- Scope bleed into Phase 2: EDA is easy to over-invest in once real data is in front of you. Timebox it — a first-pass validation that shortfall ranges are realistic is the Phase 1 bar; building the actual scenario generator is Phase 2's job.